Your Paper Title Here

Paper #XXX
Anonymous

Abstract

We present SystemName, a [hardware/software/compiler technique] that [capability]. [Problem: why existing arch/-PL/OS approaches fall short.] Our key insight is that [observation about hardware-software interaction]. SystemName exploits this through [technique], achieving [X]× speedup and [Y]% energy reduction compared to [baseline] on [benchmarks].

ACM Reference Format:
Paper #XXX. 2026. Your Paper Title Here. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 3 pages.

1 Introduction

Modern [hardware/software] systems face [challenge] due to [trend] [1]. While prior work has addressed [related problem] [2], [gap remains]. This paper addresses the [arch/-PL/OS] challenge of [specific problem].

Key Insight. We observe that [insight about hardware-software interaction]. This observation is supported by our analysis of [N] benchmarks on [hardware platform] (Section 2).

Approach Overview. SystemName addresses this through [technique]. Unlike [prior approach], SystemName [key distinction], enabling [benefit].

We make the following contributions:

- We identify and characterize [problem] through analysis of [benchmarks/hardware] (Section 2).
- We propose SystemName, a [technique] that [capability] (Section 3).
- We implement SystemName in [context: compiler/hardware/OS] and evaluate on [benchmarks] (Section 5).
- We demonstrate [X]× [speedup/efficiency] improvement over [state-of-the-art], with only [Y]% area/power overhead.

2 Background and Motivation

2.1 Hardware/Software Context

Describe relevant architecture, PL, or OS background [4].

2.2 Characterization Study

Figure 1 shows [measurement] across [benchmarks] on [hardware platform].

2.3 Opportunity Analysis

Based on this characterization, we identify [N] key opportunities:

Opportunity 1. [Description with concrete numbers.]

Performance characterization: breakdown of execution time,
cache miss rates, or energy consumption across benchmarks

Figure 1. Characterization of [metric] across [N] benchmarks on [hardware]. On average, [X]% of [time/energy] is spent on [bottleneck], motivating [your approach].

Algorithm 1 [Algorithm name] in SystemName

```
Input: computation graph G(V, E), resource constraints R
Output: mapping M : V → R
for each node v ∈ TopologicalSort(V) do
    t_v ← max_{(u,v)∈E}(t_u + latency(u))
    r* ← arg min_{r∈R} Cost(v, r, t_v)
    M[v] ← r*
end for
return M
```

Opportunity 2. [Description.] These opportunities motivate the design of SystemName.

3 Design

Figure 2 shows the overall architecture of SystemName.

3.1 [Hardware/Compiler/OS Component A]

Describe the first key component. The core scheduling algorithm is shown in Algorithm 1.

3.2 [Hardware/Compiler/OS Component B]

Describe the second component. The performance improvement from this component can be modeled as:

$$S = \frac{1}{(1-f) + \frac{f}{p} + \frac{\alpha \cdot f}{B}} \tag{1}$$

where f is the parallelizable fraction, p is the number of processing elements, B is the memory bandwidth, and α is the arithmetic intensity (ops/byte).

System architecture: hardware blocks, compiler passes, or OS mechanisms and their interactions

Figure 2. Architecture of SystemName. [Describe the key components: hardware units, compiler passes, or OS mechanisms.]

```
for (i = 0; i < N; i
    ++)
  for (j = 0; j < M;
       j++)
    C[i][j] += A[i][
        k]
                 * B[k
                     ][
                     j
                     ];
```

Before

```
for (ii = 0; ii < N;
     ii += TILE)
  for (jj = 0; jj <
       M;
        jj += TILE)
    kernel(A, B, C,
           ii, jj);
```

After (SystemName)

Figure 3. Code transformation example. SystemName converts [pattern] (left) into [optimized pattern] (right), improving [metric] by [X]×.

3.3 Example: Code Transformation

Figure 3 shows how SystemName transforms [code pattern] to exploit [hardware feature].

4 Implementation

We implement SystemName as follows:

- **[Hardware component]:** [X]K gates in [HDL], synthesized at [Y] GHz using [process node].
- **[Compiler component]:** [X]K lines of [language], integrated with [LLVM/GCC/custom compiler].
- **[OS component]:** [X] lines of kernel module in [language].

5 Evaluation

We evaluate SystemName to answer:

1. How does SystemName compare to state-of-the-art on standard benchmarks?
2. What is the hardware/software overhead?
3. How does each component contribute to the improvement?
4. How sensitive is SystemName to [key parameters]?

Table 1. Benchmark suite characteristics.

Benchmark	Domain	Instructions	Working Set
BenchA	Image	2.1B	64 MB
BenchB	NLP	4.8B	128 MB
BenchC	Graph	1.3B	256 MB
BenchD	HPC	8.2B	512 MB
BenchE	Serving	0.6B	32 MB

Table 2. Performance comparison (speedup over baseline). Higher is better. Bold indicates best result.

Benchmark	Base	Prior A	Prior B	SystemName
BenchA	1.00×	1.42×	1.55×	**2.13**×
BenchB	1.00×	1.28×	1.39×	**1.87**×
BenchC	1.00×	1.15×	1.22×	**1.64**×
BenchD	1.00×	1.51×	1.68×	**2.35**×
BenchE	1.00×	1.33×	1.41×	**1.92**×
Geomean	1.00×	1.33×	1.44×	**1.96**×

5.1 Methodology

Simulation/Hardware. We evaluate using [simulator/FPGA/real hardware]: [details].

Benchmarks. We use [SPEC CPU/PARSEC/SPLASH/MLPerf/custom] benchmarks. Table 1 summarizes the evaluation suite.

Baselines. We compare against: (1) [Baseline A] [2], (2) [Baseline B] [3], and (3) [Baseline C].

5.2 Performance Results

Table 2 shows the main performance comparison. SystemName achieves [X]× geometric mean speedup over [baseline].

5.3 Area and Power Overhead

5.4 Ablation Study

5.5 Sensitivity Analysis

We vary [key parameter] from [min] to [max] to understand its impact on performance.

Stacked bar chart: area/power breakdown by component

Figure 4. Area and power overhead of SystemName. The total overhead is [X]% area and [Y]% power, dominated by [component].

Grouped bar chart: performance with components disabled

Figure 5. Ablation study. Removing [Component A] reduces speedup from [X]× to [Y]×, confirming its importance.

6 Discussion

Generalizability. [Discuss applicability to other architectures/workloads.]

Limitations. [Honest discussion of limitations and assumptions.]

7 Related Work

[Hardware Approaches]. Prior architecture work [2, 3] addresses [problem]. SystemName differs by [distinction].

[Compiler/PL Approaches]. Compiler techniques [4] have targeted [problem]. SystemName complements these by [distinction].

[OS/Runtime Approaches]. OS-level approaches [1] provide [capability]. SystemName extends this with [technique].

8 Conclusion

We presented SystemName, a [technique] that advances [arch/PL/OS area] by [capability]. SystemName achieves [X]× speedup over state-of-the-art with only [Y]% overhead, demonstrating the effectiveness of [key insight].

References

[1] John L. Hennessy and David A. Patterson. 2019. *A New Golden Age for Computer Architecture*. Vol. 62. Communications of the ACM. 48–60 pages. Turing Award Lecture.

[2] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borber, et al. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. ACM, Toronto, ON, Canada, 1–12. https://doi.org/10.1145/3079856.3080246

[3] Hyoukjun Kwon, Parashar Chatarasi, Michael Pellauer, Angshuman Parashar, Tushar Krishna, and Paul Sarber. 2018. MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, Williamsburg, VA, 461–475.

[4] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, Palo Alto, CA, 75–86.